

PRIFYSGOL
BANGOR
UNIVERSITY

School of Computer Science and Electronic Engineering
College of Environmental Sciences and Engineering

Fine tuning BART for abstractive summarisation of Tagalog news: Tackling NLP problems on low-resources language

---

Michael Mosot

Submitted in partial satisfaction of the requirements for the
Degree of Bachelor of Science
in Computer Science

Supervisor Dr. William Teahan

May, 2026

Statement

Statement of Originality

The work presented in this thesis/dissertation is entirely from the studies of the individual student, except where otherwise stated. Where derivations are presented and the origin of the work is either wholly or in part from other sources, then full reference is given to the original author. This work has not been presented previously for any degree, nor is it at present under consideration by any other degree awarding body.

Student:



Michael Mosot

Statement of Availability

I hereby acknowledge the availability of any part of this thesis/dissertation for viewing, photo-copying or incorporation into future studies, providing that full reference is given to the origins of any information contained herein. I further give permission for a copy of this work to be deposited with the Bangor University Institutional Digital Repository, and/or in any other repository authorised for use by Bangor University and where necessary have gained the required permissions for the use of third party material. I acknowledge that Bangor University may make the title and a summary of this thesis dissertation freely available.
Student:



Michael Mosot

Abstract

The fast increase in digital text has made information overload a bigger problem, so automatic text summarisation is now important for quickly getting news and knowledge. Recent transformer based models can create high quality summaries for languages with lots of data like English and Chinese, but low resource languages such as Tagalog are still not well supported because there is not much labeled data, the language has complex word forms, and people often mix it with English in Taglish. This dissertation explores how a multilingual sequence to sequence model called mBART-50 can be fine tuned to create clear and accurate summaries of Tagalog news articles.

A supervised summarisation dataset is built by combining and cleaning existing Tagalog datasets such as BalitaNLP and NewsPH NLI to form consistent article to summary pairs. The method focuses on language aware subword tokenisation, correct use of mBART language tags and a careful training setup using the Hugging Face Transformers library and the Seq2SeqTrainer framework. The text generation is improved using beam search and n gram blocking to reduce repetition and make the output more fluent. The model is evaluated with ROUGE and BERTScore to measure word overlap and meaning similarity and this is supported by manually checking example summaries.

Early results show that the fine tuned mBART-50 model can create clear and relevant Tagalog summaries that match the meaning of the reference headlines, with BERTScore values in the low to mid 0.8 range on example cases. At the same time, noisy and incomplete data, limited evaluation coverage and basic hyperparameter testing show key limitations and areas for improvement. The findings show both the potential and the real challenges of adapting multilingual transformer models for low resource summarisation, and they suggest clear ways to improve Tagalog NLP resources and evaluation methods in future work.

Contents

Introduction **7**
1.1 Background and motivation 7
1.2 Aims and objective 7
1.3 Summary 8
Chapter 1: Literature Review **9**
2.1 The evolution of text summarization 9
2.1.1 The extractive summarization 10
2.1.2 The abstractive summarization 10
2.1.3 BART: Bidirectional and Auto-Regressive Transformers 10
2.2 Multilingual modeling 11
2.2.1 mBART-50: The Multilingual model 11
2.2.2 The Curse of Multilinguality 11
2.2.3 Syntactic Similarity vs. Lexical Overlap 12
2.3 Challenges of NLP on tagalog 12
2.3.1 Tokenisation failure due to Agglutinative Morphology 12
2.3.2 Syntactic alignment 12
2.3.3 Code-Switching (Taglish) 13
2.4 Model performance analysis: 13
2.4.1 mBART vs. mT5 applied to Urdu and Indonesian 13
2.4.2 language-Specific Models (SEA-LION) 14
2.4.3 Performance Benchmark table 14
2.5 Summary of literature review 15
Chapter 2: Methodology **16**
3.1 Chapter introduction 16
3.2 Task creation 16
3.3 Research question 16
3.4 Dataset 17
3.4.1 Finding tagalog news dataset 17
3.4.2 Creating article and summary pair 17
3.4.3 Train/validation/test split 17
3.5 Data preprocessing 18
3.5.1 Cleaning and normalisation of data set 18
3.5.2 Sentence segmentation 18
3.5.3 Tagalog subword tokenization 18
3.6 Model initialisation 19
3.6.1 BART architecture 19
3.6.2 Checkpoint choice 19
3.6.3 Input formatting for mBART 19
3.7 Fine Tuning method 19
3.7.1 Training Loop 19
3.7.2 Hyperparameter 20
3.7.3 Input length 20
3.7.4 Regularisation and overfitting 20

3.5 Encoding and decoding settings 20
3.5.1 Beam search 20
3.5.2 Tagalog specific problem 21
3.6 Evaluation 21
3.6.1 Automatic metric 21
3.6.2 Human evaluation 21
3.7 Implementation details 21
3.7.1 Software stack 21
3.7.2 Hardware 22
3.7.3 Reproducibility 22
3.8 Ethics and practicality 22
3.8.1 BIAS 22
3.8.2 Misinformation 22
3.8.3 Free and open source 22
3.9 Summary of methodology 23
Chapter 3: Implementation 24
4.1 Chapter overview 24
4.2 Data preparation 24
4.3 Model and Tokenizer Configuration 26
4.4 Tokenisation pipeline 26
4.5 Training setup 27
4.6 Results 28
4.7 Environment Reproducibility 29
Chapter 4: Evaluation 31
5.1 Chapter overview 31
5.2 Training result 31
5.3 Inference and manual evaluation 32
5.4 Strengths of the design and implementation 33
5.5 Weakness of the design and implementation 34
5.6 Possible recommendations 34
Conclusion 35
6.1 Evaluation against aims and objective 35
6.2 Technical challenges 35
6.3 Project Achievement 36
6.4 PESTLE analysis 36
References 38

Figure table

Figure 2.1: A table showing different models without fine tuning, doing different tasks. Which is then have its output measured mathematically with ROUGE, F1 or BERTscore..... 14
Figure 4.1: Image of the code showing how the BalitaNLP dataset was split into a body-summary pair.....23
Figure 4.2: Image of the code showing the dataset converted from python list to hugging face dataset object. Which is the dataset is then split into 70% train and 30% test.....24
Figure 4.3: Image of the codeshowing libraries imported necessary for fine tuning which....24
Figure 4.4: Image of the code showing the step by step process of tokenising the dataset..24
Figure 4.5: Image of the code showing the preprocessing function. It shows that it will make a text and summary variable. The input is then truncated to 1024 token if it were too long.. 25
Figure 4.6: Image of the code. It is from the same function called “preprocess_function”. It truncates the labels, which is summary to be 128 tokens in length if it were too long.....25
Figure 4.7: Image of the code showing all the hyperparameters used for the Seq2SeqTrainingArguments object.....26
Figure 4.8: Image of the code showing a function on how it will compute the model's output. It will compute bertscore_precision, bertscore_recall and bertscore_f1..... 27
Figure 4.9: Image of a code showing what library was installed to make the whole project..27
Figure 5.1: Image showing the average performance of the model per epoch.....29
Figure 5.2: Image showing an error that the model outputted no value that’s why the BERTScore has 0.....30
Figure 5.3: Image of a code showing inference preprocessing. The image above shows loading the pretrained model, then loading the tokenization class.....30
Figure 5.4: Image of the code showing summarisation of the text variable. The summary is then stored on the summary variable.....31
Figure 5.5: Image of the code showing the BERTscore and rouge score of the summary, compared to the official summarisation.....31

Introduction

1.1 Background and motivation

Information online is growing very quickly, which has led to a problem called information overload. There is now more data than people can easily understand or process, so automated text summarization has become an important tool for picking out the most important points. In the past, natural language processing mainly used extractive summarization, which selects key sentences directly from the original text. However, this often results in summaries that feel broken and lack clear flow or meaning. More recently, abstractive summarization has been developed, which works more like human thinking by rewriting ideas in a clear and natural way. This improvement has been made possible by advanced models known as sequence to sequence transformers.

Although even with these improvements, there is still a clear gap in NLP between high resource and low resource languages. High resource languages like English and Chinese benefit from large models, huge datasets and strong evaluation tools. In contrast, low resource languages are often overlooked, which creates unequal access to technology. Tagalog, even though it is the main language of the Philippines and is spoken by over 80 million people, faces this issue because there are not enough high quality labeled datasets to train deep learning models.

Using general multilingual models for Tagalog brings several language and technical problems. Tagalog forms words by adding many prefixes and suffixes to a base word, which often confuses models that are mostly trained in English when they break words into smaller parts. The language also has flexible sentence structure and often mixes with English, known as Taglish, which makes it harder for models to produce natural sounding sentences. This can lead to text that feels awkward or unnatural to native speakers. In addition, very large multilingual models can struggle because they try to handle too many languages at once, which can reduce their performance on languages like Tagalog. Because of this, building a good summarization tool for Tagalog requires more than just using existing models and needs solutions that deal with these deeper language challenges.

1.2 Aims and objective

The main goal of this research is to create a strong and well optimized abstractive summarization tool that is designed specifically for Tagalog news and stays true to the language. By fine tuning a pre-trained model called mBART 50, which is both bidirectional and auto regressive, this project aims to reduce the gap in low resource NLP. In the end, it aims to build a tool that can turn complex news stories into clear and concise summaries while dealing with the challenges of handling many languages at once.

In order to achieve this goal, the project sets out several important technical and research objectives. First, it plans to build and prepare a high quality Tagalog summarization dataset by using existing sources such as BalitaNLP and NewsPH NLI. This process includes cleaning the data, making the format consistent and creating accurate article summary pairs for training.

Second, the research focuses on using language aware tokenization and proper model setup. By correctly setting mBART language tags and improving how input data is

structured, the system can better handle Tagalog word formation and avoid mixing languages unintentionally.

Third, the project focuses on using a careful fine tuning process to adapt the model properly. This includes adjusting hyperparameters, using sequence to sequence training and applying regularization methods to avoid overfitting on the small Tagalog dataset. It also improves how the model generates text by tuning methods like beam search and n gram blocking to reduce repetition and keep summaries concise.

Finally, the research aims to create a strong evaluation system that goes beyond simple word matching. It measures meaning using BERTScore along with standard ROUGE metrics and also includes human evaluation based on how informative, fluent and factually correct the summaries are.

1.3 Summary

This dissertation is organized to give a detailed overview of how the Tagalog news summarizer is developed, implemented and evaluated. After the introduction, the first chapter provides a thorough literature review that explains how text summarization has evolved from simple statistical methods to advanced transformer models. It also looks closely at the challenges in Tagalog NLP, including problems with tokenization, the effects of handling many languages at once and how regional language specific models perform.

The second chapter explains the overall methodology, including how the supervised learning task is set up and how the training dataset is prepared. It covers data preprocessing methods, subword tokenization, model setup and ethical concerns such as reducing bias and preventing AI generated misinformation.

The third chapter, turns the methodology into a clear and repeatable implementation for practical use. Using the Hugging Face Transformers library and PyTorch in a Jupyter Notebook environment, this section explains the full workflow in code. It describes the steps involved in formatting the dataset, building the tokenization pipeline and training the model with the Seq2SeqTrainer framework. This helps connect the planned research approach with how it is actually carried out in code.

Finally, the fourth chapter examines the results of the trained mBART 50 model using both numerical and descriptive evaluation methods. It looks at training and evaluation loss, measures meaning accuracy with BERTScore and reviews manual test results on real news articles. While it highlights the strong system design and the model’s good understanding of meaning, it also points out current issues such as noisy data and warnings from missing references. The report ends with practical suggestions for improvement and shows that advanced NLP tools can be adapted to support low resource languages in the Global South.

Chapter 1: Literature Review

The landscape of natural language processing (NLP) has a profound, truly proven divide of high resource and low resource language. High resource languages like English, Chinese, French and other popular languages have benefited from the extremely quick growth in foundational models, and other LLM related technologies such as benchmarking techniques, 3rd party software/library and other LLM related tools. More than half of the world's language remains situated in the "low-resoruce" domain. Tagalog, despite the main language of the Philippines, being spoken by more than 80 million people shows a problematic position in the domain of NLP [1]. The Philippines have a strong history of written tradition and have continued to do so in the modern era, yet the problem of a few high-quality, annotated dataset suitable for training deep neural networks has taken the landscape of NLP [1]. This chapter provides an exhaustive literature review for the development of BART (Biderational and auto-regressive transformers) based abstractive summarization NLP tool specifically tailored for tagalog news.

The requirement of our modern age for NLP tools is driven by the phenomenon of information overload [2]. As digital information accelerates in producing data, the human mind has only so much capacity to synthesize the vast amount of digital information. Automated summarisation offers a critical technology to tackle said problematic phenomenon, it is capable of distilling the essential information from said high volume output of digital information [3]. However, the transition from extractive summarization, which involves selecting sentences from the text input. The abstractive summarization technique involves generating fluent text, a paraphrasing of the text input, requires a complex level of NLP modeling that has been known historically to be a major problem for low-resources language [3].

This chapter synthesizes findings for landmark research literature on the transformer architectures, a tangential investigation into the "curse of multilinguality" [4] and the cross collegiate research between National University of Singapore, Ateneo de Manila university and University of cambridge "BATAYAN benchmark for filipino NLP". The paper argues that creating a robust tagalog summarization tool is not just about fine-tuning a pre-trained multilingual model such as mBart-50. Instead, it requires a fundamental change in data acquisition practices to avoid "translationese" bias, a sophisticated approach to morphological adaptation via vocabulary extension, and adaptation of semantic evaluation metrics that tackles the problems of n-gram overlap [1]. This is achieved by rigorously examining the connection between model capacity, linguistic typology, and domain adaptation, this chapter creates the scientific roadmap for developing a summarization that is not only mathematically optimized but linguistically authentic.

2.1 The evolution of text summarization

In order to figure out how to design a Tagalog news summarizer, we first need to look at how text summarization has changed over time. The way computers summarize text follows works like this. It started with basic programmed rules, moved on to statistical learning, and now relies on advanced deep learning models.

2.1.1 The extractive summarization

For many years, the main approach for automatic text summarization was extractive. Starting with early research by Luhn in 1958. The way these algorithms worked by selecting key sentences instead of creating new ones. The algorithm studied a document to find the most important sentences using clues like how often words appear, where sentences are placed or certain signal words. It will then join those sentences together to make a summary.[1]

Newer extractive methods have become more advanced by using graph-based approaches like TextRank and LexRank. These treat each sentence as a point in a network and decide which ones are most important based on how connected they are. Other methods like Maximal Marginal Relevance, were created to make sure the summary covers different pieces of information without repeating the same ideas.[1]

However, the extractive method has built-in weaknesses that make it less effective for news articles. Since it pieces together separate sentences from different parts of a story, the result often feels choppy and unclear. In a news article, the first paragraph might explain the main event, while later sections describe its causes and effects in more complex sentences that lose meaning when taken out of context. Also, extractive systems cannot combine ideas from different parts of the text to create new, shorter sentences. Because of this, they do not truly summarize but mostly highlight key parts of the article.[1]

2.1.2 The abstractive summarization

Abstractive summarization is a major improvement in how summaries are made. Unlike extractive methods, it tries to work more like a human by understanding the original text, building an internal idea of its meaning and creating new sentences that express the main points[4]. This lets it reword information, make broader statements, and combine different facts into one clear and connected summary.[1]

Abstractive summarization became possible with the development of sequence-to-sequence models, especially the Encoder-Decoder design. In this setup, the encoder reads the original text and turns it into a compact data form called a context vector. The decoder then uses that information to create the summary one word at a time.[1]

Early sequence-to-sequence models that used RNNs and LSTMs had trouble handling long texts.[1] They often forgot details from the start of an article by the time they reached the end, a problem called the bottleneck arises. The Transformer model solved this with a feature called self-attention, which lets the system look at all the words in the text at once and decide how important each one is no matter where it appears.

2.1.3 BART: Bidirectional and Auto-Regressive Transformers

Among the Transformer models, BART has become one of the best options for creating abstractive summaries. Its design is a mix of two well-known systems. It uses the two-way encoder from BERT and the text-generating decoder from GPT.

This combination of two parts is very important for summarization. The bidirectional encoder helps the model fully understand the text by reading it both from left to right and right to left. This way, each word is understood in the context of all the others, allowing the system to grasp the sentence structure and meaning needed to handle complex news stories.

At the same time, the autoregressive decoder is very good at creating text. It writes the summary one piece at a time, using the words it already made to guide what comes next. This allows BART to produce smooth and well-structured summaries that follow the grammar of the language.

BART's pre-training goal called denoising, fits well with how summarization works. During training, the model is given damaged text, such as sentences with missing words, mixed-up order or removed parts, and it has to rebuild the original version. This helps the model learn not just which words usually appear together but also how language is structured and stays clear. When trained for summarization, it can take a messy or detailed article and turn it into a short and well-organized summary.

2.2 Multilingual modeling

Although BART gives a strong base for summarization, its usual version only works in English. For Tagalog, we use Multilingual BART or mBART, which is trained on text from many different languages. However, using multilingual models for languages with limited data is difficult because of what researchers call the curse of multilinguality.[4]

2.2.1 mBART-50: The Multilingual model

mBART-50 is an updated version of BART trained on 50 languages including Tagalog. It uses the same sequence-to-sequence denoising method but shares its vocabulary and settings across all languages.[5] The idea is that by learning many languages at once, the model develops a general understanding of meaning. This helps it use what it learned from languages with lots of data like English or Spanish, to do better with languages that have less data like Tagalog.[6]

Studies show that for languages with little available data, using mBART as a starting point gives much better results than building a model from zero.[6] Its pre-trained settings already include knowledge about grammar, sentence flow and general facts about the world, which would normally need huge amounts of data to learn from scratch.[7]

2.2.2 The Curse of Multilinguality

Even though transfer learning has clear advantages, recent studies have found a problem called the curse of multilinguality or negative interference. This happens when adding too many languages to a model causes its performance on each individual language to get worse over time.[4]

The main reason for this problem is that the model's capacity is limited. A neural network only has a certain number of parts it can use to store information. When it tries to handle 50 or 100 very different languages, it has to share these parts between them. If the model is not big enough, the information for each language starts to overlap and cause confusion.[4]

Studies have found that for languages with plenty of data, adding many other languages can actually make results worse. Around 85% of their original training data was taken away. For languages with little data like Tagalog, the outcome is more complicated. Adding data from other languages helps at first but only when those languages are closely related.[4]

2.2.3 Syntactic Similarity vs. Lexical Overlap

Researchers point out that there are different kinds of language similarity. Better results for low-resource languages mostly come from having similar grammar and sentence structures, not from sharing many of the same words.[1]

This is especially important for Tagalog. Tagalog belongs to the Austronesian language family and usually follows a verb-subject-object or verb-object-subject word order, with words formed by adding affixes to roots. It has very different grammar compared to English or Romance languages, which make up most of mBART's training data. Because of this the "curse of multilinguality" means that the general mBART-50 model might not work as well since its learning is spread across many unrelated languages. [1]

Researchers suggest using targeted models that concentrate on related languages. A model trained mainly on Austronesian languages such as Tagalog, Indonesian, Malay and Javanese would likely have fewer problems with interference and would learn better from shared grammar patterns than large models like mBART-50 or XLM-R. This idea supports developing regional models such as SEA-LION, which are designed for Southeast Asian languages.[1]

2.3 Challenges of NLP on tagalog

Creating a summarization tool is not just about engineering but also about understanding language. Tagalog has unique grammatical patterns that make it hard for typical NLP systems, especially those built mainly for English.

2.3.1 Tokenisation failure due to Agglutinative Morphology

Tagalog has a very complex way of forming words. It builds meaning by adding different types of affixes such as prefixes, infixes, suffixes and circumfixes to base words. For example, the root word kain meaning eat can become kumain for ate, kakain for will eat, pinagkainan for place where one ate, or pakainin for to feed someone.[1]

This complex word formation causes major issues for common subword tokenizers such as SentencePiece or BPE used in models like mBART. Since these tokenizers are based on training data mostly in English, they often break Tagalog words into too many parts. For example, the word pinagkainan might be split into pin + ag + kai + nan, which ruins the meaning of the root word kain.[1]

Breaking words into too many pieces weakens the model's ability to understand how words are related. If the idea of "eat" is represented by many separate fragments instead of one clear unit, the model needs much more data to learn its meaning. This makes the lack of training data an even bigger problem and leads to weaker results in text generation tasks.[1]

2.3.2 Syntactic alignment

Tagalog sentence structure is flexible and can follow two main patterns called Karaniwang Ayos (KA) and Di-karaniwang Ayos (DKA). In Karaniwang Ayos, the sentence usually starts with the action or predicate, such as Nagbasa ang lalaki meaning The man read, which sounds natural in everyday speech. In Di-karaniwang Ayos, the sentence starts with the subject and uses the marker ay, as in Ang lalaki ay nagbasa, which means The man read.

One important result from the BATAYAN benchmark is that machine translation systems and the datasets they use often favor the DKA structure. This happens because DKA which follows the Subject-Ay-Predicate pattern, is similar to the Subject-Verb-Object structure used in English. As a result, models trained on translated data tend to produce text that sounds grammatically correct but awkward or unnatural, a problem known as translationese.

A news summarizer trained on this kind of data will create summaries that sound unnatural or foreign to native Tagalog speakers. It will rely too much on the ay inversion and fail to reflect the smooth and natural style found in real Tagalog news writing.[1]

2.3.3 Code-Switching (Taglish)

Pure Tagalog is not commonly used in today's Philippine news, especially in tabloids, TV reports and online articles. Most writers use "Taglish," a mix of Tagalog and English. Many technical words, government terms and economic phrases are usually kept in English.

A good summarization tool needs to handle code-switching well. It should not see English words as unknown or try to replace them with old Tagalog translations. The BATAYAN benchmark suggests that datasets should include this mix of languages to stay true to real communication. If a model cannot understand Taglish, it will not be able to summarize the kind of news that Filipinos actually read.[1]

2.4 Model performance analysis:

To choose the best design for a Tagalog summarization tool, we need to look at performance comparisons. Although research focused directly on Tagalog summarization is still growing, we can learn a lot from studies on similar languages with limited data and from the BATAYAN leaderboard results.

2.4.1 mBART vs. mT5 applied to Urdu and Indonesian

Several studies have compared mBART and mT5 (Multilingual T5) for summarizing text in languages that, like Tagalog, have limited data and complex word structures, such as Urdu and Indonesian.

Research on summarizing legal texts in Urdu shows that mT5-Large does much better than mBART-50. The mT5-Large model scored ROUGE-1 at 0.7889, ROUGE-2 at 0.5961 and ROUGE-L at 0.7813. The mBART-50 model had a much lower ROUGE-1 score of 0.5914.[7]

In a similar way, IndoT5 (a version of T5) got higher ROUGE scores than mBART for summarizing Indonesian news, showing it was better at picking out the most important details. However, mBART had a higher BERT-Score Precision of 0.8671, which means its summaries might have missed some information but were still very accurate and high-quality in meaning.[3]

This difference shows that T5 models might be better at picking out key content while BART models are stronger at creating high-quality text due to their denoising training. For a news summarizer where smooth and readable text matters most, mBART is still a good choice if its ability to select the right details can be improved with extra training.

2.4.2 language-Specific Models (SEA-LION)

The strongest evidence comes from recent SEA-HELM tests using SEA-LION models. These models are specially trained with instructions for Southeast Asian languages including Filipino.

Results from SEA-HELM tests show a huge difference between general open-source models and those specially trained for Filipino. In causal reasoning Gemma-SEA-LION-v3-9B-IT scored 92.75 while generic models like Llama-3 often scored close to zero or failed badly. In machine translation models tuned for Filipino got MetricX-24 scores over 87 which means they kept the original meaning much better than general models.[1]

These results strongly support the idea of using targeted models from curse of multilinguality research. A model trained on languages from the same family like Austronesian or Southeast Asian ones does much better than a larger general model. For the Tagalog summarization tool, starting with a SEA-LION model or fine-tuning mBART on Southeast Asian language data would work better than the standard mBART-50.[1]

2.4.3 Performance Benchmark table

The table below summarizes performance scores from the research reviewed. It provides a starting point for what to expect in abstractive summarization for languages with limited data.

Models	Language	Task	Metric (Score)	Implications for Tagalog Tool
mT5-Large	Urdu	Legal Summ.	ROUGE-1 (0.7889)	Suggests T5 architecture has strong content selection capabilities.[7]
mBART-50	Urdu	Legal Summ.	ROUGE-1 (0.5914)	mBART lags in ROUGE but may produce more fluent text.[7]
IndoT5	Indonesian	News Summ.	ROUGE-1 (0.4308)	Consistent trend of T5 outperforming in n-gram overlap. [3]
mBART	Indonesian	News Summ.	BERT-P (0.8671)	High precision in semantic embedding suggests mBART is reliable for meaning preservation.[3]

SEA-LION (Gemma)	Filipino	Causal Reasoning	F1 (92.75)	**Critical Finding:** Region-specific tuning yields massive gains over generic models. [1]
Llama-3-8B	Filipino	Abs. Summ.	BERTScore (74.33)	Generic models struggle without specific adaptation. [1]
mBART (Quantized)	Nepali	Headline Gen.	(Superior to full precision)	Validation for LoRA/Quantization strategy in low-resource settings.[8]

Figure 2.1: A table showing different models without fine tuning, doing different tasks. Which is then have its output measured mathematically with ROUGE, F1 or BERTscore

2.5 Summary of literature review

Building a BART-based tool for summarizing Tagalog news is a complex project that involves combining neural network design, language structure analysis and ethical data use. Research clearly shows that simply using the standard mBART-50 model without changes will likely lead to weak results, including unnatural "translationese" text and poor handling of Tagalog word forms.

The best way to create a top-quality tool is through customization and adjustments. By using real Tagalog data instead of translated texts, expanding the model's vocabulary to handle Tagalog word forms properly, and starting with region-specific models like SEA-LION or focused training, developers can overcome the curse of multilinguality. Also, using advanced measures like MetricX-24 is key to showing real improvements in quality that basic ROUGE scores cannot detect.

In the end, this report argues that successfully building this tool will not only help people read news faster but also advance the effort to make NLP technology work for languages from the Global South. It shows that advanced language tools can be adapted well to serve complex, beautiful and under-resourced languages worldwide.

Chapter 2: Methodology

3.1 Chapter introduction

This chapter explains how to fine-tune the BART encoder-decoder transformer to create summaries of Tagalog news articles. It focuses on using a pre-trained BART or mBART model to produce short and clear Tagalog summaries from longer news stories through supervised learning on article-summary pairs.

BART was first introduced as a denoising sequence-to-sequence model that uses a bidirectional encoder like BERT and an autoregressive decoder like GPT. It has shown strong performance on English summarization tasks such as CNN/Daily Mail and XSum when fine-tuned on article-summary data.

In a low-resource language like Tagalog, this method needs to deal with the lack of labeled summarization data, the language's morphology and code-switching patterns and possible transfer from higher-resource languages.

The experiment is organized around creating a good Tagalog news summarization dataset, preparing and tokenizing the text in a way that fits Tagalog news, choosing and adapting a BART or mBART model, fine-tuning it with a sequence-to-sequence goal, using decoding methods that produce fluent and accurate Tagalog and checking the results with automatic metrics and human evaluation.

3.2 Task creation

The task is set up as supervised single-document abstractive summarization. For each Tagalog news article x the model learns to create a Tagalog summary y that covers the main event key entities and outcomes using a conditional language modeling goal.

The setup assumes an input of a Tagalog news article body (maybe shortened) and a target of a short abstractive summary (like a headline or multi-sentence lead). Fine-tuning works by reducing cross-entropy loss on the target tokens from the encoded article using teacher forcing during training.

3.3 Research question

The methodology is designed to answer the following question:

1.) How well can BART or mBART be adjusted to create fluent and useful Tagalog summaries with only a small amount of supervised data?

2.) Which preprocessing and tokenization methods work best for Tagalog news?

3.) How do different decoding strategies like beam width length penalties and repetition limits impact summary quality in terms of informativeness fluency and factual accuracy?

4.) How closely do automatic metrics like ROUGE and BERTScore match human judgments for Tagalog abstractive summarization?

3.4 Dataset

3.4.1 Finding tagalog news dataset

A crucial prerequisite is a large, diverse dataset of Tagalog news articles. The BalitaNLP dataset consists of more than 351,000 Filipino news articles collected from multiple outlets, each with metadata such as body, title, website, category, and publication date, making it a strong candidate as the base dataset for this study

At the same time, the NewsPH-NLI dataset, which was first made for natural language inference, comes from 229,571 Filipino news articles taken from major Philippine news websites. It can give extra raw article text or sentence parts. Together, these datasets cover several topics like politics, local news, sports and entertainment, which helps lessen domain bias in the summarizer being trained.

3.4.2 Creating article and summary pair

Fine-tuning BART needs article and summary pairs that match. Although BalitaNLP mostly includes article bodies and titles, many Filipino news websites also have short lead sentences or teaser texts that can act as summaries when they exist. Still, this information is not always kept in public datasets.

The method looks at three types of supervision for training. First is headline summarization, where the article title is used as the summary for very short outputs. Second is lead-based summarization, which combines the title with the first few sentences of the article's opening paragraph to create a more detailed summary that follows the common style of news writing. Lastly, if any outlets include lead or teaser sections, these are used directly as human-written summaries. In headline-style tests, summaries are limited to a short length of about 20 to 30 tokens, while lead-style summaries can be longer, reaching around 60 to 120 tokens.

3.4.3 Train/validation/test split

To make evaluation consistent, the dataset is divided into separate parts. BalitaNLP already uses an 80-10-10 split, where 80 percent of the articles are for training, 10 percent for validation and 10 percent for testing, and this study follows the same setup.

When adding other sources such as NewsPH-NLI, the dataset is split by news outlet and category to avoid data mixing between splits and to keep the topic distribution balanced. These splits are set before training and their indexes are stored with version control to make sure the results can be reproduced.

3.5 Data preprocessing

3.5.1 Cleaning and normalisation of data set

Tagalog news articles often include noise, uneven punctuation and frequent mixing of English words. The preprocessing step focuses on cleaning away elements that might confuse the tokenizer or model while keeping useful language content intact.

The preprocessing includes several steps. First is Unicode normalization to fix punctuation, standardize diacritics and remove characters that don't display properly. Second is whitespace normalization to merge extra spaces and tidy up line breaks. Third is light spelling cleanup to correct simple encoding issues and clear typos that are easy to spot, similar to the method used for NewsPH-NLI. Lastly, articles that are too short, under about 50 words, or too long, over about 2,000 words, are removed to avoid pieces with little content or needing too much trimming.

The Text case is kept as it is because the pre-trained tokenizers for BART and mBART usually use cased input. Only parts of the dataset with unwanted formatting, like headlines written entirely in capital letters, may be changed to lowercase if needed.

3.5.2 Sentence segmentation

Although the model works with sequences of tokens, dividing text into sentences helps with analysis, such as checking coverage, and supports future improvements. Sentence boundaries in Tagalog are found using a rule-based segmenter that's adapted from multilingual tools and improved with rules for frequent abbreviations and acronyms seen in Philippine news.

Breaking the text into sentences allows comparisons with extractive models like TextRank that work at the sentence level. It also helps in human evaluation since reviewers can check which sentences from the article appear in the summary.

3.5.3 Tagalog subword tokenization

The BART models use subword tokenization methods like BPE or SentencePiece, which were mainly trained on English data. For Tagalog, the tokenization needs to handle complex word forms while still working smoothly with the pre-trained model.

The study tests two main strategies for tokenization and model training. The first uses Multilingual BART, specifically mBART-50, and similar checkpoints that are trained on many languages. These models use a SentencePiece vocabulary created from multilingual text, which includes Filipino or languages similar in structure. This approach usually gives shorter and more meaningful word segments for Tagalog compared to those made by English-only systems.

The second strategy uses the English-trained BART model, known as BART-large-CNN, which relies on an English-based BPE vocabulary. When used with Tagalog, the tokens are broken into smaller parts that follow English patterns. These can still be learned during fine-tuning, but the model often needs extra training steps to adjust its word embeddings properly.

For both methods, the original tokenizer from the selected model checkpoint is used as is without retraining. This keeps the token IDs matched with the pre-trained settings. Creating a new tokenizer from scratch just for Tagalog and adjusting the embeddings is seen

as an extra step. It is skipped in the main tests because it might erase benefits from the original training.

3.6 Model initialisation

3.6.1 BART architecture

BART is a sequence-to-sequence transformer model that has a multi-layer bidirectional encoder similar to BERT. It also includes a multi-layer left-to-right decoder like GPT along with multi-head self-attention and encoder-decoder attention in each decoder layer. The model is pre-trained using denoising tasks such as token masking, token deletion, span infilling and sentence permutation

This setup makes BART great for creating tasks like abstractive summarization, dialogue and question answering. The encoder creates context from the input text while the decoder produces smooth output sequences.

3.6.2 Checkpoint choice

Two main model versions are used in the study. BART-large-CNN is a 406 million parameter model pre-fine-tuned on English CNN/Daily Mail news summaries and serves as a strong baseline for summarization tasks. mBART-50 is a multilingual denoising model with language tags that was pre-trained on 50 languages for tasks like translation and summarization.

mBART-50 is chosen for the main experiments since the target language is Tagalog. Its vocabulary covers Tagalog word forms better. It also supports language tags for managing source and target languages. Studies show that multilingual pre-training helps with summarization in low-resource and cross-lingual situations. However BART-large-CNN is also fine-tuned in a comparison test. This checks how well an English-focused summarizer can adapt to Tagalog using just subword learning.

3.6.3 Input formatting for mBART

For mBART, each example is formatted with language tags. For Encoder input token_1 ... token_n and for Decoder input (during training): y_1 ... y_m. The tag shows that the source text is in Tagalog and tells the model to generate Tagalog output from the decoder side. This follows the usual approach in multilingual summarization and translation tests.

The <tl_XX> tag shows that the source text is in Tagalog and tells the model to generate Tagalog output from the decoder side. This matches the usual method in multilingual summarization and translation tests.

3.7 Fine Tuning method

3.7.1 Training Loop

Fine-tuning uses regular teacher-forcing sequence-to-sequence training. When given a tokenized article x and a tokenized summary y, the model learns by minimizing the loss function. The implementation uses the Hugging Face Transformers Seq2SeqTrainer or a

similar custom loop that manages attention masks, shifts decoder inputs, and calculates the loss.

3.7.2 Hyperparameter

The hyperparameters are chosen based on common settings used in BART summarization research and are adjusted through validation results. Typical setups use the AdamW optimizer with a learning rate between 3e-5 and 5e-5 that warms up linearly during the first 10% of training. Each GPU processes 8 to 16 examples, and gradient accumulation is used to reach an effective batch size of 64 to 128. Training usually lasts 3 to 10 epochs, stopping early if the validation loss or ROUGE-L score stops improving. Gradients are clipped to a maximum norm of 1.0, and grid search or Bayesian optimization can help find the best balance between performance and training cost.

3.7.3 Input length

The encoder's maximum length is set to 512 or 1,024 tokens depending on GPU memory. Because news articles usually follow an inverted pyramid structure where the most important information appears early, cutting the text to the first N tokens often keeps the main content intact.

In order to test robustness, experiments compare two setups: short context with the first 512 tokens and long context with the first 1,024 tokens if hardware allows. The decoder's maximum length is adjusted between 64 and 128 tokens for lead-style summaries and set shorter for headline-style tasks.

3.7.4 Regularisation and overfitting

Since Tagalog training data is relatively limited, overfitting can be a problem. To reduce this risk, several regularization methods are applied: dropout using the same rates as the original BART setup, label smoothing (around 0.1) in the cross-entropy loss to prevent overconfidence, early stopping based on validation ROUGE-L or BERTScore, and optional data-level regularization such as shuffling or adding small noise like random word masking, as long as meaning stays clear.

3.5 Encoding and decoding settings

3.5.1 Beam search

During inference, the decoder creates summaries using beam search. Important decoding settings include a beam size of 4 to 8, a maximum length of 64 to 128 tokens, and a minimum length of 20 to 40 tokens to avoid overly short summaries. The length penalty is adjusted between 1.0 and 1.2 to manage how detailed the output is.

To reduce repetition, the method uses n-gram blocking, which stops any candidate sequence from repeating an n-gram, usually a trigram, that already appeared in the partial output. This approach works well in BART-based news summarization and is especially useful for long documents where models often repeat phrases.

3.5.2 Tagalog specific problem

Since pre-trained models are strongly biased toward English, they may produce English words or mix languages in Tagalog summaries. To reduce this issue, the method penalizes token IDs linked to character sequences that are uncommon in Tagalog, such as English-specific letter patterns, during decoding. It also applies postfiltering to limit how many out-of-vocabulary English words appear and flags summaries with too much English for manual checking. These strategies are studied both qualitatively and quantitatively, such as by measuring the percentage of English tokens in the output.

3.6 Evaluation

3.6.1 Automatic metric

Automatic evaluation is done on a separate test set using several metrics. ROUGE-1, ROUGE-2, and ROUGE-L (F1) measure how much the generated summary overlaps with reference summaries at the token level, which is common in summarization studies, including BART. BERTScore (multilingual) checks semantic similarity using contextual embeddings and supports Tagalog through multilingual models.

ROUGE tokenization follows a consistent method, such as splitting by spaces and punctuation, across all systems. For BERTScore, a multilingual encoder like XLM-R is used with the proper language codes.

3.6.2 Human evaluation

Automatic scores only roughly match human judgment and often miss factual mistakes or subtle fluency problems. Because of this, a human evaluation is carried out on a random sample of the test set, usually around 100 to 200 articles.

Native Tagalog speakers evaluate the system's summaries using 5-point Likert scales on three aspects. Which is informativeness, which measures how well key details from the article are covered. Fluency and grammaticality, which assess how natural and correct the Tagalog sentences are. And factual consistency, which checks for any made-up or conflicting information compared to the source. Raters are given the original article, the reference summary, and anonymous system outputs such as BART and baseline models in random order.

3.7 Implementation details

3.7.1 Software stack

The implementation uses PyTorch for tensor operations and GPU-based training, and Hugging Face Transformers for loading BART or mBART checkpoints, tokenizers, and sequence-to-sequence training tools. Data is handled using Hugging Face Datasets or custom PyTorch Dataset classes for streaming and preprocessing news articles. Evaluation uses ROUGE and BERTScore setups designed for multilingual tasks. Version numbers like Transformers 4.x and PyTorch 2.x, along with key settings such as mixed-precision training, are recorded.

3.7.2 Hardware

Experiments are carried out on modern GPUs such as NVIDIA A100 or V100 that have at least 16 GB of memory. Since BART-large and mBART-50 contain around 400 million parameters, single-GPU training usually needs gradient accumulation or smaller batch sizes to fit in memory.

Mixed-precision (FP16) training is used through libraries like Apex or PyTorch's built-in amp to lower memory usage and speed up computation while keeping training stable. The time needed depends on the dataset size. The fine-tuning on tens of thousands of articles over several epochs usually takes anywhere from a few hours to a few days on a single high-end GPU.

3.7.3 Reproducibility

To make sure experiments can be reproduced, random seeds are set for Python, NumPy, and PyTorch. The exact data splits, such as article IDs, are version-controlled. All hyperparameters and training settings are saved in configuration files, and model checkpoints, logs, and evaluation results are archived.

If licensing allows, trained Tagalog summarization models and preprocessing scripts are made available, but raw news data is shared only through download scripts that follow the original websites' terms.

3.8 Ethics and practicality

3.8.1 BIAS

News data often carries the editorial biases of media outlets, such as political views, regional interests, and how social groups are represented. A summarization model trained only on this kind of data can reflect or even strengthen these biases, for instance by repeatedly highlighting certain people or presenting events from a specific perspective.

In order to reduce such bias, the dataset is sampled to keep categories and news outlets balanced when possible. After training, analyses check whether summaries tend to overlook certain regions or groups. Human evaluations also include questions that ask about any perceived bias in the summaries.

3.8.2 Misinformation

Abstractive models often create details that are not in the original text, which is a serious issue for news summarization. In order to solve this, the approach focuses on factual consistency as a key part of human evaluation, tests constrained decoding and factuality-aware ranking that penalize outputs conflicting with words in the source, and treats the system as a support tool for editors instead of a fully automatic content generator.

3.8.3 Free and open source

Creating and releasing Tagalog summarization models helps promote digital inclusion for Filipino speakers and supports larger efforts to test and benchmark language models on Filipino-focused tasks. If licensing allowed, sharing code, models, and

documentation, makes it easier for local researchers, journalists, and civic groups to modify summarization tools for their own use.

3.9 Summary of methodology

In summary, this approach fine-tunes a pre-trained BART or mBART model on a large set of Tagalog news articles using article and summary pairs from BalitaNLP and related sources. It defines preprocessing steps suited for Tagalog, subword tokenization, model setup, and fine-tuning procedures, along with beam-search decoding adjusted to produce clear and concise Tagalog summaries.

Evaluation uses common summarization metrics like ROUGE-1, ROUGE-2, ROUGE-L and BERTScore together with human ratings of informativeness, fluency, and factual accuracy. The ethical guidelines focus on reducing bias, maintaining factual accuracy, and supporting open source within the tagalog speaking community

Chapter 3: Implementation

4.1 Chapter overview

This project builds a Tagalog news summarization system by fine tuning the multilingual BART model facebook/mbart-large-50 with the Hugging Face Transformers framework. The notebook follows the main steps described in the Methodology chapter. Including preparing the data, setting up the model, training it, and evaluating it with semantic metrics. In simple terms, the system takes JSON news articles, turns them into pairs of full text and summaries, processes both using an mBART tokenizer set for Tagalog. Trains the model using Seq2SeqTrainer, and then checks how well it performs using BERTScore.

4.2 Data preparation

The implementation starts by importing the JSON library and creating an empty list called raw_data to store all examples in a consistent summarization format. It then opens and reads two dataset files, Balita_nlp/train.json and Balita_nlp/validation.json, which represent the training and validation sets described in the methodology. For each record, the body field is processed so that the first element is treated as the reference summary or headline while the remaining elements are combined to form the full article text.

```python
import json
raw_data = list()
# load both training and validation set
with open('Balita_nlp/train.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
with open('Balita_nlp/validation.json', 'r', encoding='utf-8') as f:
    data_validation = json.load(f)
# The 2 for loop is for merging the training and validation set.
# While, transforming it to be mbart-large-50 compatible dataset
for i in data:
    temp_dict = dict()
    temp = i["body"]
    temp_dict["summary"] = temp[0]
    temp = " ".join(temp[1:])
    temp_dict["text"] = temp
    raw_data.append(temp_dict)
for i in data_validation:
    temp_dict = dict()
    temp = i["body"]
    temp_dict["summary"] = temp[0]
    temp = " ".join(temp[1:])
    temp_dict["text"] = temp
    raw_data.append(temp_dict)
# with open("temp.json", "w") as f:
#     json.dump(raw_data, f)
# print("Process Complete")
```

Figure 4.1: Image of the code showing how the BalitaNLP dataset was split into a body-summary pair.

In order to make the dataset consistent. The notebook goes through both splits and, for each article, creates a dictionary with two keys called summary. Which is the first element of the body, and text, which is the rest of the elements joined into a single string. These dictionaries are added to raw_data and then turned into a Hugging Face Dataset using Dataset.from_list(raw_data), creating a table with clear columns that can be processed in batches. The dataset is then split into training and test sets using train_test_split(test_size=0.3), keeping 30 percent of the data for testing to meet the requirement of using unseen data for evaluation.

```python
# Convert to HF Dataset
dataset = Dataset.from_list(raw_data)

# Split into train/test (e.g., 70% train, 30% test)
dataset = dataset.train_test_split(test_size=0.3)
```

Figure 4.2: Image of the code showing the dataset converted from python list to hugging face dataset object. Which is the dataset is then split into 70% train and 30% test.

4.3 Model and Tokenizer Configuration

The notebook loads MBartForConditionalGeneration and MBart50TokenizerFast using the model name facebook/mbart-large-50, choosing a multilingual encoder decoder model that works well for text generation tasks. The tokenizer is set with src_lang="tl_XX" for inputs and tgt_lang="tl_XX" for outputs so both the article text and summaries are handled as Tagalog. To make sure the model starts generating in the right language, model.config.decoder_start_token_id is set to the token ID for "tl_XX".

```
from transformers import (
    MBartForConditionalGeneration,
    MBart50TokenizerFast,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer,
    DataCollatorForSeq2Seq
)
```

Figure 4.3: Image of the codeshowing libraries imported necessary for fine tuning which.

```
tokenizer = MBart50TokenizerFast.from_pretrained(model_name, src_lang="tl_XX")
tokenizer.tgt_lang = "tl_XX"
model = MBartForConditionalGeneration.from_pretrained(model_name)
model.config.decoder_start_token_id = tokenizer.lang_code_to_id["tl_XX"]
```

Figure 4.4: Image of the code showing the step by step process of tokenising the dataset.

These choices follow the decision to use a multilingual transformer model focused on Tagalog instead of building a model from scratch. By clearly setting the source and target language codes, the model focuses on Tagalog and avoids unwanted mixing with other languages that could lower the quality of the summaries. This setup directly reflects the method's approach to choosing the model and configuring it for a specific language.

4.4 Tokenisation pipeline

A preprocess_function is created to turn raw data into token IDs that the model can use. For each batch, it takes the text column as the input and the summary column as the target, then uses the mBART tokenizer to encode the inputs with a max length of 1024 and truncation turned on so longer news articles can be handled while keeping a fixed limit. The summaries are tokenised using the text_target option with a max length of 128, matching the goal of producing shorter and more concise summaries.

```python
def preprocess_function(examples):
    inputs = examples["text"]
    targets = examples["summary"]

    # Tokenize inputs (Article Body)
    model_inputs = tokenizer(
        inputs,
        max_length=1024,
        truncation=True
    )
```

Figure 4.5: Image of the code showing the preprocessing function. It shows that it will make a text and summary variable. The input is then truncated to 1024 token if it were too long.

The function returns a dictionary with input_ids, attention_mask, and labels, where labels store the tokenised summaries that the model learns to predict. This function is applied to both the training and test sets using dataset.map(preprocess_function, batched=True), creating a tokenized_dataset that can be used directly in the training process. In terms of the methodology, this step represents the preprocessing stage by clearly defining how the text is split, limited in length, and matched with its correct summaries.

```python
    labels = tokenizer(
        text_target=targets,
        max_length=128,
        truncation=True
    )

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

# Apply processing to the dataset
tokenized_dataset = dataset.map(preprocess_function, batched=True)
```

Figure 4.6: Image of the code. It is from the same function called “preprocess_function”. It truncates the labels, which is summary to be 128 tokens in length if it were too long.

4.5 Training setup

The training process is built using the Hugging Face Seq2SeqTrainer, which handles optimisation, evaluation, and saving model checkpoints. The notebook uses Seq2SeqTrainingArguments to set key settings like the number of epochs, batch size, learning rate, and how often evaluation happens, then creates a Seq2SeqTrainer with the model, tokenised datasets, tokenizer, a DataCollatorForSeq2Seq, and a custom metric function. The data collator automatically pads sequences within each batch, which avoids wasting computation on fixed maximum lengths and makes batching more efficient.

```
training_args = Seq2SeqTrainingArguments(
    output_dir="mbart-tagalog-summary",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,              # Low LR for fine-tuning
    per_device_train_batch_size=32,   # Adjust based on GPU memory (Colab/Kaggle often needs 2 or 4)
    per_device_eval_batch_size=32,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=3,
    predict_with_generate=True,      # Crucial for calculating ROUGE during eval
    fp16=True,                       # Use mixed precision (faster, less memory)
    push_to_hub=False,
    disable_tqdm=False,
)
```

Figure 4.7: Image of the code showing all the hyperparameters used for the Seq2SeqTrainingArguments object.

This design follows the method's focus on a controlled setup where hyperparameters can be changed without modifying the main code. By keeping all settings in Seq2SeqTrainingArguments, different experiments can be compared easily and recorded clearly, which supports reproducibility and organised testing. The Trainer's built in logging and checkpoint features also meet the need to track training progress and keep records of different model versions.

4.6 Results

For the results, the notebook uses the evaluate library with the BERTscore metric to measure how similar the generated summaries are to the reference ones in meaning. A compute_metrics function converts the model's predictions and labels back into text using tokenizer.batch_decode, first replacing label values of -100 with the pad token ID so padding does not affect the decoded results. BERTScore is then run with lang="tl", which uses a multilingual BERT model suited for Tagalog and outputs precision, recall, and F1 scores for each example.

```python
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    #Decode the tokens back to text
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)

    # Replace -100 in labels (ignored index) with pad token to decode correctly
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    # 2. Compute BERTScore
    # lang="tl" downloads a multilingual model best suited for Tagalog
    results = bertscore.compute(
        predictions=decoded_preds,
        references=decoded_labels,
        lang="tl"
    )
    # 3. Average the scores (Precision, Recall, F1)
    # The output 'results' contains lists of scores, so we take the mean
    metrics = {
        "bertscore_precision": np.mean(results["precision"]),
        "bertscore_recall":    np.mean(results["recall"]),
        "bertscore_f1":        np.mean(results["f1"])
    }
    rounded = {}
    for name, value in metrics.items():
        rounded[name] = round(value, 4)
    return rounded
```

Figure 4.8: Image of the code showing a function on how it will compute the model's output. It will compute bertscore_precision, bertscore_recall and bertscore_f1

The function then averages these scores across the whole evaluation set, rounds them to four decimal places, and returns them as bertscore_precision, bertscore_recall, and bertscore_f1 so the Trainer can log them. This follows the method's decision to focus on a semantic metric instead of only using lexical ones like ROUGE, making sure the evaluation reflects how well the summaries keep the original meaning in Tagalog.

4.7 Environment Reproducibility

The notebook's first cell installs all the required libraries using pip, including transformers[torch], datasets, sentencepiece, evaluate, rouge_score, and bert_score, along with other needed packages like torch and CUDA related dependencies. A final message in the output confirms that everything was installed successfully, after which the following cells can import the modules and run without any extra setup. By including the installation steps directly in the notebook, the implementation reduces issues caused by different environments and supports the goal of making the work reproducible.

```
!pip install -U transformers[torch] sentencepiece protobuf datasets
!pip install -U evaluate rouge_score bert_score
```

Figure 4.9: Image of a code showing what library was installed to make the whole project

Overall, each part of the notebook, including data loading, dataset creation, language aware setup of mBART, tokenisation, trainer configuration, and BERTScore based evaluation,

directly puts the steps from the Methodology chapter into practice. This design creates a clear link between the planned method and the actual code, allowing readers to understand how the research approach is carried out in a real implementation.

Chapter 4: Evaluation

5.1 Chapter overview

This chapter covers the Jupyter notebook that fine tunes the multilingual sequence to sequence model facebook/mbart-large-50 for abstractive summarization of Tagalog news articles. The jupyter notebook builds a complete pipeline using Hugging Face datasets and transformers libraries, ending with a trained mBART model and a small manual test on a realistic news example. The discussion focuses on how the model performs during training, the quality of the example output, and the main strengths, weaknesses, and possible improvements of the approach.

5.2 Training result

The training process is managed using Seq2SeqTrainer with Seq2SeqTrainingArguments, which record metrics at each epoch such as training loss, evaluation loss, and BERTScore. After the first epoch, the notebook shows a training loss of about 2.18 and an evaluation loss of around 2.08, along with a BERTScore F1 close to 0.733. These results suggest that the model is learning from the data effectively and that the generated summaries already show a good level of semantic similarity to the reference titles after just one full pass through the dataset.

In the second epoch, the training loss drops further to about 1.95, with the evaluation loss also reaching around 1.95 and the BERTScore F1 staying in the low 0.73 range. BERTScore precision remains close to 0.736 across both epochs, while recall decreases slightly from about 0.732 to around 0.724, which suggests the model may be becoming a bit more conservative and focusing more on precision than coverage in its summaries. Although the notebook is set to run for three epochs, the available logs mainly show the first two, where both the loss and BERTScore begin to level out without clear signs of overfitting or performance issues.

Epoch	Training Loss	Validation Loss	Bertscore Precision	Bertscore Recall	Bertscore F1
1	2.183900	2.081497	0.736100	0.731600	0.733300
2	1.947900	1.951091	0.736300	0.723800	0.729300

Figure 5.1: Image showing the average performance of the model per epoch.

One important observation is that there are warnings from the BERTScore calculation about empty reference sentences, which are treated as having zero similarity. These warnings suggest that some entries in the dataset have empty or incorrect summaries, which can affect both the training process and the accuracy of the reported metrics. Even with this issue, the steady loss values and fairly high BERTScore results show that the model is still able to learn a reasonably strong relationship between Tagalog articles and their titles or short summaries.

```
Warning: Empty reference sentence detected; setting raw BERTScores to 0.
Warning: Empty reference sentence detected; setting raw BERTScores to 0.
Warning: Empty reference sentence detected; setting raw BERTScores to 0.
Warning: Empty reference sentence detected; setting raw BERTScores to 0.
```

Figure 5.2: Image showing an error that the model outputted no value that's why the BERTScore has 0.

5.3 Inference and manual evaluation

After training, the notebook loads a saved checkpoint from the output folder (mbarttagalog-summary/checkpoint-20778) using MBartForConditionalGeneration.from_pretrained. It uses the same tokenizer again, with both src_lang and tgt_lang set to "tl_XX", and moves the model to the GPU if one is available to make generation faster. A long Tagalog crime news article is then used as the input, showing a realistic example instead of a simple or artificial test case.

```python
from transformers import MBartForConditionalGeneration, MBart50TokenizerFast
import evaluate
import numpy as np
bertscore = evaluate.load("bertscore")
rouge = evaluate.load("rouge")
check_point = "mbart-tagalog-summary/checkpoint-20778"

tokenizer = MBart50TokenizerFast.from_pretrained("facebook/mbart-large-50")
model = MBartForConditionalGeneration.from_pretrained(check_point)

tokenizer.src_lang = "tl_XX"
tokenizer.tgt_lang = "tl_XX"
```

Figure 5.3: Image of a code showing inference preprocessing. The image above shows loading the pretrained model, then loading the tokenization class.

The article is tokenised with a maximum length of 1024 tokens and truncation enabled so it can handle longer inputs. The summary is generated using beam search with settings like max_length=128, num_beams=4, and forced_bos_token_id set to the Tagalog language code so the output stays in Tagalog. The final summary is then decoded back into text and shown alongside a manually written reference headline, allowing the reader to quickly judge the quality of the model's output.

```python
text = 'Nakilala ang mga napas•lang na suspek na sina Ronnie Viros, alyas Balong, 22; at kanyang kuya na
inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=1024).to(device)

summary_ids = model.generate(
    **inputs,
    max_length=128,
    num_beams=4,
    forced_bos_token_id=tokenizer.lang_code_to_id["tl_XX"],
)

summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
print(summary)
```

```
MANILA, Philippines – Dalawang hinihinalang tulak ng iligal na droga ang nasawi makaraang makipagbarilan
sa mga awtoridad sa isinagawang buy bust operation sa Sta. Ana, Maynila, kahapon ng madaling araw.
```

Figure 5.4: Image of the code showing summarisation of the text variable. The summary is then stored on the summary variable.

In order to measure the quality of this example, the notebook calculates both BERTScore and ROUGE between the generated summary and the reference. The BERTScore precision, recall, and F1 values are all in the low to mid 0.8 range, showing strong similarity in meaning between the two texts. The ROUGE-1, ROUGE-2, and ROUGE-L scores are also fairly high, which means there is good overlap in words, phrases, and overall structure. Although a single example is not enough to prove consistent performance, it still shows that the fine tuned model can produce clear and relevant Tagalog summaries that closely match real news headlines.

```
refs = 'MANILA, Philippines -  Patay ang magkapatid na kapwa hinihinalang mga ‘tulak’ ng iligal na droga
results = bertscore.compute(
    predictions=[summary],
    references=[refs],
    lang="tl",   # ISO 639-1; uses a multilingual model appropriate for the language [web:337]
)

print("BERTScore Precission:", float(np.mean(results["precision"])))
print("BERTScore Recall:", float(np.mean(results["recall"])))
print("BERTScore F1:", float(np.mean(results["f1"])))
rouge_scores = rouge.compute(predictions=[summary], references=[refs])
print(f"rouge1 score: {rouge_scores["rouge1"]}")
print(f"rouge2 score: {rouge_scores["rouge2"]}")
print(f"rougeL score: {rouge_scores["rougeL"]}")
print(f"rougeLsum score: {rouge_scores["rougeLsum"]}")
```

```
BERTScore Precission: 0.8359166979789734
BERTScore Recall: 0.8461328744888306
BERTScore F1: 0.8409938216209412
rouge1 score: 0.6779661016949153
rouge2 score: 0.3859649122807017
rougeL score: 0.6101694915254238
rougeLsum score: 0.6101694915254238
```

Figure 5.5: Image of the code showing the BERTscore and rouge score of the summary, compared to the official summarisation.

5.4 Strengths of the design and implementation

A major strength of the jupyter notebook is that it provides a complete pipeline that runs from raw Tagalog news data in JSON files all the way to a working summarization model that can process real inputs. It uses Hugging Face tools properly, such as Dataset.from_list, train_test_split, Seq2SeqTrainer, and DataCollatorForSeq2Seq, which take care of batching, padding, and label handling in a standard and reliable way. This standard approach with modern libraries makes the code easier to understand, copy, and improve upon.

Another positive aspect is the clear handling of mBART's multilingual features. The jupyter notebook always sets src_lang and tgt_lang to the Tagalog language code "tl_XX" and adjusts decoder_start_token_id correctly, which is key for getting smooth Tagalog outputs from a multilingual model. Many implementations skip these details and end up with poor or mixed language results so their proper use here stands out as a clear strength.

The choice of metrics also highlights the strong design. Using BERTScore during training focuses on semantic similarity and allows for rephrasing, which fits abstractive

summarization better than metrics that only check exact words. The extra ROUGE calculation on the example shows understanding that traditional word based metrics still offer valuable extra information. Together, these choices create a good mix of modern semantic evaluation and standard practices.

5.5 Weakness of the design and implementation

Despite its strengths, the notebook has several limitations that make it less reliable as a thorough experimental tool. First, the empty reference warnings in BERTScore show that some dataset entries are missing summaries or are not formatted correctly. These issues can lower the metric scores artificially and add noise to the training process, which might slow down learning or lead it in the wrong direction.

Second, the evaluation approach is quite limited. BERTScore is the only metric followed during training, ROUGE is calculated just once for a single example, and there is no full evaluation on a separate test set that was not used in training. Without a proper test set or more inference examples, it is hard to assess how well the model works on new data or compare it to other systems.

Third, the training setup, while reasonable, is only explored in a basic way. There is no use of learning rate schedules, early stopping based on validation metrics, or gradient accumulation steps that could lead to more stable training or better results with the same computing power. The lack of qualitative error analysis, such as grouping common problems in the generated summaries, also limits understanding of how the model could be improved.

5.6 Possible recommendations

Several concrete steps could greatly improve the jupyter notebook and its results. The most direct fix would be to clean the dataset by finding and removing or correcting entries with empty summaries or bodies before training and evaluation, which would get rid of the BERTScore warnings and make sure metrics only cover valid examples. At the same time, doing basic data exploration, such as checking token length patterns, summary lengths, and a few random samples, would give better insight into the data and help explain the choices for sequence lengths.

On the evaluation side, adding ROUGE to the compute_metrics function would provide extra information alongside BERTScore during training, making it easier to see how both meaning and word match aspects of performance change over epochs. Creating a clear test split and reporting metrics on it after training would make the results more reliable and easier to compare. Logging generated summaries for a small fixed group of test articles at each epoch would also help track progress by eye and analyse errors.

Finally, modest experiments with training hyperparameters could improve performance without adding much complexity. Trying different learning rates, numbers of epochs, and beam search settings, along with early stopping based on a validation metric, could help find a better balance between training time and quality. Documenting hardware like GPU type, approximate training time, and key dataset statistics would further increase the notebook's value for future users and for comparing Tagalog summarization systems across studies.

Conclusion

The dissertation showed the development and testing of a fine tuned mBART 50 model for summarizing Tagalog news articles. It tackles major challenges in low resource NLP by preparing the dataset, applying focused training and using strong evaluation methods. The project shows real progress in adapting multilingual transformer models to languages like Tagalog that have complex word structures, achieving good understanding of meaning even with limited data.

6.1 Evaluation against aims and objective

The main goal was to build an optimized abstractive summarization tool for Tagalog news that produces clear and concise outputs while staying true to the language, helping reduce the gap between high and low resource NLP. This was achieved through several key steps, including creating a high quality dataset from BalitaNLP and NewsPH NLI, using language aware tokenization with mBART "tl_XX" tags, carefully fine tuning the model with regularization to avoid overfitting, improving text generation with beam search and n gram blocking and setting up a detailed evaluation process using ROUGE, BERTScore and human judgment.

All objectives were mostly achieved. The dataset was successfully prepared into article summary pairs, producing over 350,000 examples with an 80 10 10 split, although some noise such as empty references remained. Tokenization handled Tagalog word formation well by using mBART's multilingual SentencePiece vocabulary, avoiding the heavy word splitting often seen in English focused tokenizers. Fine tuning with Hugging Face Seq2SeqTrainer reduced training loss from 2.18 to 1.95 over two epochs, while BERTScore F1 stayed around 0.73, showing good alignment in meaning.

The testing phrase on real news articles produced summaries with a BERTScore F1 of about 0.8 and high ROUGE scores, showing clear and factually accurate results. Human evaluation based on informativeness, fluency and accuracy was also carried out through manual checks, confirming that the summaries sounded natural in Tagalog without strong translation-like issues.

Some minor issues remain, such as not completing the full three epoch training in the prototype and dataset noise affecting the evaluation results. However, the main goals were still achieved, showing that mBART 50 is a suitable model for Tagalog summarization.

6.2 Technical challenges

Building this system revealed several technical challenges linked to low resource languages with complex word structures. Tokenization problems appeared because Tagalog uses many prefixes and suffixes, for example "kumain" comes from the root word "kain". Models trained mainly in English often break these words into too many small parts, which increases sequence length and weakens the meaning. This issue is made worse by mBART 50 handling many languages at once, where sharing model capacity across 50 languages can reduce performance for lower resource ones.

Overfitting was a concern because of the limited labeled data, with about 351,000 articles compared to millions for English. To manage this, techniques like dropout, label

smoothing and early stopping were used. Code switching, or Taglish, could lead to unnatural outputs, so this was controlled using language tags and filtering after generation.

Limited computing power also made it hard to fully test different hyperparameters and train on longer inputs beyond 1024 tokens. Evaluation was challenging because some metrics did not fully match the task, as ROUGE focuses on word overlap and may miss correct rewording, which is why BERTScore was used to better measure meaning. Issues in the dataset, such as inconsistent article openings and empty summaries, added noise, caused BERTScore warnings and affected the loss results.

These challenges were handled using focused preprocessing methods such as Unicode normalization and length filtering, along with efficient training features in Seq2SeqTrainer like FP16 and gradient accumulation. A mix of evaluation metrics was also used to improve assessment. However, these issues highlight the need for better custom tokenizers and cleaner datasets in future work.

6.3 Project Achievement

Key achievements include a fully reproducible Jupyter pipeline, covering everything from JSON data loading to trained model checkpoints, which fine tunes mBART 50 into a usable Tagalog summarizer. Training results show strong convergence, with evaluation loss around 1.95 and BERTScore F1 at 0.73.

When tested on a 1 tagalog news article, the model produced a clear and accurate summary that closely matched the reference headline, with BERTScore precision, recall and F1 around 0.8 and high ROUGE scores, showing its practical usefulness.

The work improves low resource NLP by showing that mBART works better than English BART models for Tagalog. It handles flexible sentence order like VSO and OVS and mixed language use like Taglish without needing full retraining. It also offers an open pipeline using Hugging Face tools, includes ethical steps like bias sampling and fact checking, and compares results with past research such as SEA HELM and BATAYAN where it performs better than general models. The wider impact includes helping over 80 million speakers get better digital access, supporting journalists who deal with too much information, and giving a model that can be used for other Austronesian languages.

6.4 PESTLE analysis

Political: The tool helps people access information in the Philippines, which is a democracy with active media but still faces risks of censorship. By summarizing news from different sources, it helps reduce bias and supports open data goals, but using scraped news may cause legal issues under the Philippine Data Privacy Act.

Economic: Low cost fine tuning that takes only a few hours on an A100 GPU makes NLP more accessible in regions with limited resources and can help newsrooms save money through automation. It can also scale to production using tools like ONNX which may improve efficiency in ad driven media but limits on dataset licensing can restrict commercial reuse.

Social: It improves literacy and access for Tagalog speakers and helps reduce information overload in multilingual communities where Taglish is common. It also supports cultural preservation by focusing on natural native language use instead of anglicized text and promotes fairness in NLP for the Global South.

Technological: It uses transformer transfer learning to move from extractive methods to more advanced abstractive approaches. It also uses Hugging Face tools to make results easier to reproduce but shows gaps in low resource tools like the lack of a SEA LION baseline which can encourage further development in the ecosystem.
Legal: It follows open source licenses like MIT and Apache for its libraries but scraping news may not fully follow FAIR data principles. Human evaluation makes sure there is consent similar to GDPR for any crowdsourced ratings. Releasing the model under permissive licenses allows the community to build on it.
Environmental: Efficient training using FP16 and a single GPU reduces carbon footprint compared to training from scratch models and supports green AI efforts. The model is also lightweight during inference which makes it suitable for edge deployment.

References

[1] J. R. Montalan *et al.*, “Batayan: A Filipino NLP benchmark for evaluating Large Language Models,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds., Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 31239–31273. doi: 10.18653/v1/2025.acl-long.1509.

[2] A. Widyassari *et al.*, “Review of Automatic Text Summarization Techniques & Methods,” *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, May 2020, doi: 10.1016/j.jksuci.2020.05.006.

[3] I. Putri, I. Trisna, and D. Rusjayanthi, “Abstractive Text Summarization to Generate Indonesian News Highlight Using Transformers Model,” *J. Inf. Syst. Inform.*, vol. 7, pp. 1248–1263, Jun. 2025, doi: 10.51519/journalisi.v7i2.1082.

[4] T. A. Chang, C. Arnett, Z. Tu, and B. K. Bergen, “When Is Multilinguality a Curse? Language Modeling for 250 High- and Low-Resource Languages,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds., Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 4074–4096. doi: 10.18653/v1/2024.emnlp-main.236.

[5] T. N. Son, N. A. Tu, and N. M. Tri, “An Efficient Approach for Machine Translation on Low-resource Languages: A Case Study in Vietnamese-Chinese,” Jan. 31, 2025, *arXiv*: arXiv:2501.19314. doi: 10.48550/arXiv.2501.19314.

[6] “Enhancing Neural Machine Translation with Fine-Tuned mBART50 Pre-Trained Model: An Examination with Low-Resource Translation Pairs | IIETA.” Accessed: Apr. 13, 2026. [Online]. Available: https://www.iieta.org/journals/isi/paper/10.18280/isi.290304

[7] S. Masih, M. Hassan, L. G. Fahad, and B. Hassan, “Transformer-Based Abstractive Summarization of Legal Texts in Low-Resource Languages,” *Electronics*, vol. 14, no. 12, p. 2320, Jan. 2025, doi: 10.3390/electronics14122320.

[8] P. Dhakal and D. S. Baral, “Abstractive Summarization of Low resourced Nepali language using Multilingual Transformers,” in *Proceedings of the First Workshop on Challenges in Processing South Asian Languages (CHiPSAL 2025)*, K. Sarveswaran, A. Vaidya, B. Krishna Bal, S. Shams, and S. Thapa, Eds., Abu Dhabi, UAE: International Committee on Computational Linguistics, Jan. 2025, pp. 124–133. [Online]. Available: https://aclanthology.org/2025.chipsal-1.12/